AIM 2627 C++ Assessment 01

哨兵重启计划：数据链路值班夜

2026–2027

题面修订：2026 年 10 月 29 日

目录

0 这份题面怎么读 2

序章 3

Q1 定点遥测规范化（8 分（原始权重）） 4

Q2 设备标签维护（10 分（原始权重）） 6

Q3 窗口异常检测（10 分（原始权重）） 8

Q4 严格命令事务（12 分（原始权重）） 10

Q5 网格邻域风险图（12 分（原始权重）） 12

Q6 目标优先级选择（14 分（原始权重）） 14

Q7 热量与弹仓仿真（16 分（原始权重）） 16

Q8 遥测帧综合流水线（18 分（原始权重）） 18

B1 挑战：有符号十进制解析（+10 分） 20

B2 挑战：原地稳定压缩（+10 分） 22

提交与验收 24

0 这份题面怎么读

本文件是全部任务、接口、评分与提交要求的正式规格说明。环境与启动命令见学生模板 README。铜金色故事框解释为什么要做这件事，正文的编号规范才是验收依据。样例演示典型流程，不能替代完整规范；隐藏测试只检查公开规范中的行为。

本份只考查基础语法与过程式数据处理。允许用不含成员函数的普通 struct 表示记录；不要求也不允许引入自定义 class、构造函数、访问控制、继承、多态或资源所有权设计。标准库 string、vector、array 和流按已提供用法使用，不要求解释其内部类实现。函数外壳已给出，先完成函数体。B1 的引用输出、B2 的指针数组为选做桥接。

每题先读背景和任务，再看固定接口与规范，最后用样例复核更新顺序。返回值样例是函数调用示意，不是要求从 stdin 读取的文本。每份单文件自检成功时输出 PUBLIC 2/2；它表示两组公开调用通过，不表示正式满分。

题目总览

编号	题目	核心考点	原始权重
Q1	定点遥测规范化	整数运算、条件分支、字符串拼接	8
Q2	设备标签维护	字符串查找、替换、增删、完整字段边界	10
Q3	窗口异常检测	连续窗口、下标、时间复杂度	10
Q4	严格命令事务	解析、分支、事务式更新、统计	12
Q5	网格邻域风险图	二维数组、嵌套循环、边界与异常	12
Q6	目标优先级选择	遍历筛选、多级比较、整数宽度	14
Q7	热量与弹仓仿真	循环仿真、更新顺序、资源计数	16
Q8	遥测帧综合流水线	范围验证、去重、分类统计、编号整理与数据守恒	18
B1	挑战：有符号十进制解析	字符处理、引用输出、溢出预防（选做）	+10
B2	挑战：原地稳定压缩	数组指针、原地处理、稳定性（选做）	+10

运行、输入与输出

C++17、仅标准库、零硬件依赖，不读写外部文件。所有实现通过参数、返回值或对象状态交互，不自行读取 stdin，不在实现中输出调试内容。没有另行规定的越界输入不用于验收；明确规定的无效输入、异常类型、端点和失败状态都是契约。

CPP-Classroom 每题运行完整.cpp，已包含固定类型、实现区、公开自检和 main。只修改 BEGIN IMPLEMENTATION 与 END IMPLEMENTATION 之间的内容，保留标记、原文件名与接口。可在区内增加普通辅助函数；Assessment 2 可增加必要私有成员。正式 grader 抽取实现后，结合教师固定类型重新编译，不依赖学生 main。

各题独立调用，不依赖其他题、上一测试或全局可变状态。若题目有持续状态，它应属于本次对象，而不是不同对象共享的变量。异常只在题目要求时抛出，bool 失败不能随意改成异常。

序章

序章

2027 年，赛季还没有正式开始，AIM 实验室已经开始为下一次出发倒数。上一场比赛的复盘停在一个令人尴尬的画面：地图上目标很多，终端里日志刷得很快，哨兵却始终待在原地。屏幕上的“运行正常”，显然不是赛场上的“可以工作”。

国庆的 Python 训练已经把哨兵控制流程拆成了感知、导航、决策和任务；现在轮到 C++ 接手其中更接近设备的数据链路。队长没有要求你一晚上写出完整机器人，只是把一批真实工作中常见的问题放到了桌上：乱序和重发的记录、不可信的数值、失败了一半的命令、说不清为什么发生的射击请求。

你是这晚的数据链路值班工程师。先让每条输入有明确去处，再让每次更新有明确规则，最后交出一份能够复核的回放汇总。所有场景都是离线仿真，零硬件依赖；故事为任务提供动机，验收以正文编号规范为准。

Q1　定点遥测规范化　*(8 分（原始权重）)*

背景

2027 年联赛出发前一晚，实验室里终于响起了熟悉的开机提示音。有人松了口气，有人盯着屏幕上 103% 的血量和 -7% 的电量，问了一句：“它这是活了，还是回光返照?”
机器人上的不同模块并不总在同一时刻完成初始化。旧缓存可能比新配置先到，串口收到的原始值也可能超出显示范围。之前的 Python 哨兵模块已经能发送通知，现在战队准备把关键链路迁到 C++。你的第一项工作，就是让上位机收到一条含义明确、格式稳定的自检报告。报告中的每个分号都是协议的一部分，不能因为终端里看着差不多就随意修改。

你的任务

实现 `ratio` 和 `report` 两个普通函数。前者把原始血量转换为整数百分比，后者把编号、血量和电量拼成一条自检记录。调用方直接传入参数并接收返回值；本题不要求交互输入，也不要求你设计任何自定义类型。

考点： 整数运算、条件分支、字符串拼接。实现文件：`src/main/A1_q1.cpp`。

固定接口与数据含义

```
long long ratio(long long hp, long long max_hp);
std::string report(int id, long long hp, long long max_hp, int battery);
```

接口位于骨架已有的 `aim::q1` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. hp、max_hp 的绝对值均不超过 10^{12}；id 在 [0,9999]，battery 为任意 int。
2. max_hp≤0 时，ratio 返回 -1；此时不得进行除法。report 的 HP 字段写 INVALID，但电量仍须正常处理。
3. max_hp>0 时，先把 hp 夹到 [0,max_hp]，再计算 hp×100/max_hp 的整数部分；禁止浮点计算和四舍五入。
4. battery 先夹到 [0,100]。规范化后 battery≥60 为 OK，20≤battery<60 为 WARNING，battery<20 为 LOW。
5. report 严格返回 ID=<id>;HP=< 百分比或 INVALID>;BAT=< 规范化电量 >;STATE=< 档位 >，数字不补零，不含空格和末尾换行。
6. 输入参数不会被修改；每次调用独立，不能沿用上一次调用的结果。

样例与逐步说明

```
ratio(2, 3);                // 66
report(7, 2, 3, 20);         // "ID=7;HP=66;BAT=20;STATE=WARNING"
report(0, 4, 0, -7);         // "ID=0;HP=INVALID;BAT=0;STATE=LOW"
```

第一组血量是 $2/3$，百分比截断为 66；电量恰好 20 属于 WARNING。第二条报告的最大血量无效，不能把错误写成 0% 或让整条报告消失；负电量先归零，因而仍能输出 LOW。

Q2　设备标签维护　*(10 分（原始权重）)*

背景

上午联调时，终端把相机、惯导和雷达写成了一串设备标签：cam,imu,radar。硬件换过一轮以后，有的标签需要改名，有的设备已经拆除，还有新模块等待追加。操作手希望沿用这条简单的状态栏，不想为一次改名重写整个界面。

第一次修改很快出了问题：搜索 cam 时连 cam2 也一起改了，删除最后一台设备后又留下一个逗号。另一个脚本提前记住所有位置，把短名字换成长名字后，后面的下标全都错位。你需要按顺序维护当前文本，区分完整标签与子串，并让空名单、首尾标签和重复标签都有明确行为。调用方已经提供编辑记录，本题不需要解析文本命令。

你的任务

实现 `edit_labels`，根据 `LabelEdit` 普通数据记录依次更新逗号分隔的标签文本，返回最终字符串。from 和 to 分别表示旧标签与新标签，空字符串用于表示追加或删除；调用方的原始文本和编辑记录保持不变。

考点：字符串查找、替换、增删、完整字段边界。实现文件：`src/main/A1_q2.cpp`。

固定接口与数据含义

```
struct LabelEdit { std::string from; std::string to; };
std::string edit_labels(std::string labels, const std::vector<LabelEdit>& edits);
```

接口位于骨架已有的 `aim::q2` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. labels 为空字符串或用单个逗号连接的非空标签，初始总长度 ≤ 1000；没有首尾逗号或连续逗号。每个标签长 1..32，只含 ASCII 字母、数字和下划线，区分大小写。
2. edits 最多 200 项。每项的 from、to 为空字符串或符合上述规则的标签；所有输入保证满足这些语法要求，不要求报告格式错误，也不需要数字转换或流解析。
3. 按 edits 的顺序执行。from 为空且 to 非空时，把 to 追加到名单末尾，只有原名单非空才加分隔逗号；两者均空时不操作。
4. from 非空时，只查找当前文本中第一个与 from 完全相同的完整标签。部分子串不算命中，未找到则不修改；from 等于 to 也只处理首次命中一次。
5. 命中且 to 非空时，用 to 替换该标签；命中且 to 为空时，删除该标签及必要的相邻逗号。删除唯一标签得到空字符串，任何结果都不应有首尾逗号或连续逗号。
6. 允许重复标签，追加或改名不去重，其余标签相对顺序不变。每次编辑使用上一次编辑后的文本，不能沿用修改前的字符位置。

7. 空 edits 返回原文本；不修改调用方的文本或 edits，每次调用独立。可以使用 string 的 find、substr、replace、erase、clear 和拼接等操作，不要求实现特定搜索算法。

样例与逐步说明

```
edit_labels("cam,imu,radar", {{"imu","gyro"},{"cam",""},{"","lidar"}});
// "gyro,radar,lidar"
edit_labels("cam,cam2,cam", {{"cam","camera"},{"cam",""}});
// "camera,cam2"
edit_labels("cam", {{"cam",""},{"","imu"}}); // "imu"
```

第一组依次得到 cam,gyro,radar、gyro,radar、gyro,radar,lidar。第二组只把第一个 cam 改成长名字 camera；随后删除剩下的完整 cam，不能误删 cam2 或 camera。第三组先清空名单，再追加 imu，不应在它前面留下逗号。

Q3　窗口异常检测　*(10 分（原始权重）)*

背景

设备标签终于对齐了，下一份反馈却是：“这台哨兵平均下来挺正常，就是偶尔像突然断电。”把整场比赛的电流求平均，很容易掩盖几帧集中的异常。
电控组决定用固定长度的连续窗口做预警：只要最近 k 帧的累计值超过阈值，就标出当前帧，方便回看录像。数据里允许出现负值，它表示相对基线的偏差，而不是坏数据。比赛记录很长，若每推进一帧都从头累加整个窗口，日志还没分析完，晚上的联调就开始了。你要在保持判定精确的同时，让处理时间随帧数线性增长。

你的任务

实现 `alarms`：对长度为 k 的每个连续窗口求和，返回超过阈值的窗口右端下标。返回的是从 0 开始的帧下标，不是窗口的序号、左端下标或报警数。

考点：连续窗口、下标、时间复杂度。实现文件：`src/main/A1_q3.cpp`。

固定接口与数据含义

```
std::vector<int> alarms(const std::vector<int>& values, int k, long long threshold);
```

接口位于骨架已有的 `aim::q3` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. values 长度 $n \le 200000$，元素在 $[-10^9, 10^9]$；threshold 绝对值 $\le 10^{15}$，k 为 int。
2. $k < 1$ 或 $k > n$ 时直接返回空序列，包括空输入；不得构造不完整窗口。
3. 有效窗口依次为 $[0, k-1]$、$[1, k]$，直到 $[n-k, n-1]$，相邻窗口可以重叠。
4. 仅在窗口总和严格大于 threshold 时报警；恰好相等不报警。
5. 结果按右端下标升序排列，每个窗口最多贡献一次；输入不变。
6. 要求 $O(n)$ 时间、$O(1)$ 额外工作空间，返回结果占用的空间除外。窗口和使用足够宽的整数保存。

样例与逐步说明

```
alarms({1,2,3,4}, 2, 4);    // {2,3}
alarms({2,2}, 2, 4);        // {}
alarms({-4,-3,-2}, 2, -6);  // {2}
```

第一组窗口和依次为 3、5、7，对应右端下标 1、2、3；后两个窗口报警，因此返回 2、3。第二组恰好等于阈值，不报警。负数样例中 -5 大于 -6，不能把负总和统一视为正常。

Q4　严格命令事务　*(12 分（原始权重）)*

背景

联调群里发来一串命令："先加五，再设成八，清零，最后减二。"看上去很简单，直到你发现"设成八"后面还跟着一个没删干净的调试字段。
之前的程序一读到 SET 8 就更新数值，随后才发现尾巴不合法，却已经无法恢复原来的状态。真实机器人不应该因为一条解析失败的命令留下半套修改。为了验证这个习惯，本题先用一个累加器代替执行机构：先检查整条命令、参数和执行后的范围，全部通过以后才能提交。错误命令也必须留下计数，否则复盘时谁都说不清到底丢了多少次请求。

你的任务

实现 `execute`，按输入顺序执行命令，返回 `CommandResult`。其中 value 是最终累加器值，accepted 和 rejected 分别是成功与失败的命令条数。这个 struct 只承载数据，不需要成员方法。

考点：解析、分支、事务式更新、统计。实现文件：`src/main/A1_q4.cpp`。

固定接口与数据含义

```
struct CommandResult { long long value; int accepted; int rejected; };
CommandResult execute(const std::vector<std::string>& lines);
```

接口位于骨架已有的 `aim::q4` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. 累加器从 0 开始，最多 10000 行；一行视为一条命令，空行也算一次拒绝。
2. 仅接受大小写敏感的 ADD x、SET x、RESET，允许首尾标准流空白。ADD 和 SET 必须恰有一个参数，RESET 不允许参数。
3. x 为十进制整数，按标准流提取，且在 $[-10^6,10^6]$；多余字段、缺字段、小数尾缀、未知操作和溢出均拒绝。
4. ADD 的候选结果为原值 +x，SET 为 x，RESET 为 0。候选结果也必须在 $[-10^6,10^6]$。
5. 完整验证成功才提交候选值并把 accepted 加一；其他情况只把 rejected 加一，value 保持原值。
6. 空输入返回 {0,0,0}；任意输入都应满足 accepted+rejected=lines.size()，本次调用与其他调用独立。

字符串流提示

`std::istringstream` 把已有字符串当作输入流，提取方式与 `std::cin >>` 相同，但不会读取键盘。下面只演示一个整数与尾随内容的检查：

```
std::istringstream input("12 extra");
int number = 0;
std::string extra;
bool has_number = static_cast<bool>(input >> number);
bool has_extra = static_cast<bool>(input >> extra);
// has_number=true, number=12, has_extra=true
```

第一次提取成功只说明读出了数字；第二次仍能读出 extra，说明整行还有内容。若尾部只有空白，则无法再提取出一个词。对 `12x`，整数提取可先得到 12，随后仍能读到 x，所以也必须检查尾部。对缺失参数或超出提取类型范围的整数，数字提取本身会失败。每条命令创建自己的流，先完成命令名、参数、尾部和范围检查，再修改累加器。

样例与逐步说明

```
execute({"ADD 5","SET 8 x","RESET","ADD -2"}); // {-2,3,1}
execute({"SET 1000000","ADD 1"});            // {1000000,1,1}
```

第一组状态依次为 5、5、0、-2。带尾随字段的 SET 被整条拒绝，所以它既不改变值，也不计 accepted。第二组的参数 1 本身合法，但相加后的值超界；拒绝后仍为 1000000。

Q5　网格邻域风险图　*(12 分（原始权重）)*

背景

软件组在地图上标了几个障碍，第一次回放就发现哨兵很喜欢贴着柱子走。导航同学暂时不想引入完整代价地图，只希望先看到一张简单的风险图：一个可通行格子周围贴着越多障碍，风险就越高。
有人写了八邻域，斜对角的柱子也算进去；另一个人把地图边界当成墙，导致最外圈凭空多出风险。两份图都很像那么回事，但给下游决策的含义完全不同。本题把坐标、邻域和非法地图的处理写成明确协议。你的程序负责生成风险图，不负责寻找路径，也不能悄悄修正输入地图。

你的任务

实现 `risk`，把字符地图转换为同尺寸整数矩阵。地图用字符串数组表示，访问顺序为 grid[row][column]；返回矩阵采用相同的行列顺序。

考点： 二维数组、嵌套循环、边界与异常。实现文件：`src/main/A1_q5.cpp`。

固定接口与数据含义

```
std::vector<std::vector<int>> risk(const std::vector<std::string>& grid);
```

接口位于骨架已有的 `aim::q5` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. 地图最多 300 行、300 列。空的行数组合法，返回空矩阵；非空地图必须至少有一列且所有行等长。
2. 字符只允许 .（可通行）和 #（障碍）。零列、行长不一致或含其他字符时抛 std::invalid_argument。
3. 须先验证完整地图，再计算风险，不能因为前几行正常就忽略后续非法字符。
4. 障碍格输出 -1；可通行格输出其上、下、左、右四个方向中障碍格的数量，范围是 0..4。
5. 斜对角不属于邻居，越界位置不算障碍；地图边缘不会自动增加风险。
6. 不修改输入，不返回部分地图。有效地图要求 O(行数 × 列数) 时间。

样例与逐步说明

```
risk({".#", ".."});       // {{1,-1},{0,1}}
risk({"."});               // {{0}}
risk({".#.","#.#",".#."}); // center cell: 4
risk({"..", "."});         // throws std::invalid_argument
```

第一张图只有右上角是障碍，因此左上和右下各计一个邻接障碍，左下为 0，障碍本身为 -1。单格空地没有邻居，输出 0 而不是 4。最后一组行长不一致，不能靠补齐或截断来生成答案。

Q6　目标优先级选择　*(14 分（原始权重）)*

背景

相机终于能稳定报出装甲板了，云台却在两个敌人之间左右摇摆。视觉组说最近的应该先打，决策组说残血优先，操作手只问能不能别一帧换一个目标。
为了让离线回放能复现每一次选择，大家先约定一套确定性规则：先筛掉看不见、已被击毁和超出射程的目标，再按血量、距离、编号依次比较。这只是训练仿真中的选靶协议，不代表真实比赛的完整战术。你要实现的是同一份输入永远选出同一编号，即使目标列表换了排列顺序，平局也不能交给运气。

你的任务

实现 `select_target`，从 `Target` 普通数据记录中选择一个目标并返回它的 id。坐标表示相对当前机器人原点的偏移；射程使用欧氏距离，不是曼哈顿距离。

考点：遍历筛选、多级比较、整数宽度。实现文件：`src/main/A1_q6.cpp`。

固定接口与数据含义

```
struct Target { int id; int x; int y; int hp; bool visible; };
int select_target(const std::vector<Target>& targets, int max_range);
```

接口位于骨架已有的 `aim::q6` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. 最多 100000 个目标；id 非负且互不相同，x、y 绝对值 $\le 10^6$，hp$\in$[-100,100]，visible 为 bool。
2. max_range<0 时直接返回 -1；非负时保证 $\le 10^6$。空输入也返回 -1。
3. 仅 visible=true、hp>0 且 $x^2+y^2 \le \text{max_range}^2$ 的目标有资格参与选择；射程边界包含在内。
4. 先比较 hp，较小者优先；只有 hp 相同时才比较平方距离，较小者优先；两者都相同时才比较 id，较小者优先。
5. 没有合格目标返回 -1，否则返回目标 id，而非其数组下标。输入顺序不能作为额外优先级。
6. 要求 O(n) 时间，不修改或排序输入。平方与求和必须在足够宽的整数类型中计算。

样例与逐步说明

```
select_target({{7,3,4,20,true},{2,0,5,20,true}}, 5); // 2
select_target({{7,3,4,1,true},{2,0,0,2,true}}, 5);   // 7
select_target({{1,0,0,0,true}}, 1);                  // -1
```

第一组两个目标平方距离均为 25，恰在射程边界，血量又相等，因此较小编号 2 胜出。第二组远处目标血量更低，必须先按血量选择 7，不能让距离抢走第一优先级。第三组目标血量为 0，直接淘汰。

Q7　热量与弹仓仿真　*(16 分（原始权重）)*

背景

队里有一句联调时经常出现的话：“我明明按了开火。”可发射请求只是意图，不代表一定能发射：弹仓可能空了，热量也可能顶到上限。
更容易争论的是一帧之内的先后顺序。若先判热量再冷却，某一帧会被拒绝；若先冷却再判定，同样的原始数值可能允许射击。为了把日志里的每次请求解释清楚，本题采用一套离散时间仿真。它只描述本作业中的固定规则，不能拿赛场手册的另一套热量机制替换。最后的统计和剩余资源必须能互相对得上。

你的任务

实现 `simulate`，从给定初始弹量和热量开始，逐帧处理请求，返回 `FireResult`。shots 是实际射击数，rejected 是被拒绝的射击请求数，ammo、heat 是处理完最后一帧后的末态。

考点：循环仿真、更新顺序、资源计数。实现文件：`src/main/A1_q7.cpp`。

固定接口与数据含义

```
struct FireResult { int shots; int rejected; int ammo; int heat; };
FireResult simulate(int ammo, int heat, int cool, int shot_heat, const std::vector<int>&
    requests);
```

接口位于骨架已有的 `aim::q7` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. ammo、heat、cool、shot_heat 都在 $[0,100]$；requests 最多 10000 项，每项为 0 或 1。
2. 每帧第一步固定为冷却：heat 减去 cool，若为负则归零。
3. 请求为 0 时，本帧只执行冷却；不扣弹、不增加 shots，也不增加 rejected。
4. 请求为 1 时，仅当 ammo>0 且冷却后的 heat+shot_heat≤ 100，才实际发射；等于上限 100 允许发射。
5. 成功射击时 ammo 减一、heat 增加 shot_heat、shots 加一；失败只把 rejected 加一，弹量不变，也不增加射击热量。
6. 初始 shots=rejected=0。空请求不发生任何冷却，返回原始 ammo、heat；每次调用独立。

样例与逐步说明

```
simulate(2,90,10,20,{1,1,1}); // {2,1,0,100}
simulate(3,100,20,10,{0});    // {0,0,3,80}
simulate(1,100,0,0,{1});      // {1,0,0,100}
```

第一组第一帧从 90 冷却到 80，发射后回到 100；第二帧冷却到 90，加 20 会超限，因此拒绝；第三帧再冷却到 80 才成功。最终两次射击、一次拒绝、零弹量、热量 100。这个逐帧顺序正是需要实现的规则。

Q8　遥测帧综合流水线　*(18 分（原始权重）)*

背景

出发前的最后一次回放终于能跑完了，队长却拿着三份统计问："为什么首次有效、重复和无效记录的总数，跟输入条数对不上？"大家各自的函数都看起来正确，接起来以后却把同一条记录既算成了坏包，又算成了重复包。
通信模块已经把输入拆成了编号和血量两个整数字段，你接手的是普通数据记录，不需要再读文本。但字段能放进 int 不代表数值就可信：编号可能越界，血量可能超出接收范围。你要给每条记录唯一的去处，再输出按编号整理的清单。同编号后到的合法包不覆盖首次值，非法数据不能先占编号，最终清单也不能夹带上一批回放的结果。

你的任务

实现 `summarize`，把结构化遥测记录的范围验证、分类、去重、血量规范化和编号整理组合成一个普通函数。`Telemetry` 与 `Summary` 都是已提供的普通 struct；valid、duplicate、invalid 为互斥分类计数，total_hp 为首次合法记录规范化后的总血量，ids 为升序编号。

考点：范围验证、去重、分类统计、编号整理与数据守恒。实现文件：`src/main/A1_q8.cpp`。

固定接口与数据含义

```
struct Telemetry { int id; int hp; };
struct Summary { int valid; int duplicate; int invalid; long long total_hp; std::vector<int
    > ids; };
Summary summarize(const std::vector<Telemetry>& frames);
```

接口位于骨架已有的 `aim::q8` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. frames 最多 20000 条，每条 Telemetry 已包含 id、hp 两个 int 字段，字段可为任意 int；直接读取字段，不需要文本解析。
2. 只有 id∈[0,9999] 且 hp∈[-1000,1000] 才合法。任一字段范围不满足时计 invalid，不占用 id；验证编号后才能用它访问标记表。
3. 完整合法后才判断重复。id 已有首次合法记录时计 duplicate，不增加 valid、total_hp 或 ids。
4. 首次合法记录计 valid；把 hp 夹到 [0,100] 后累加到 long long 的 total_hp，并保存编号。
5. ids 按编号升序返回，每个编号只出现一次，不按到达顺序返回；可以排序，也可以按编号顺序扫描标记表。
6. 每条记录恰好属于首次合法、合法重复、非法三类之一，始终满足 valid+duplicate+invalid=frames.size()、valid=ids.size()。空输入所有计数和总血量为 0，ids 为空。

7. 不修改 frames，每次调用独立，不依赖其他题的实现或源文件。本题单独编译和评分。

样例与逐步说明

```
Summary s = summarize({{2,200},{1,-5},{2,10},{-1,30}});
// s.valid=2, s.duplicate=1, s.invalid=1
// s.total_hp=100, s.ids={1,2}
Summary t = summarize({{1,-1001},{1,30},{1,1001}});
// t.valid=1, t.duplicate=0, t.invalid=2
// t.total_hp=30, t.ids={1}
```

第一组编号 2 的首次 hp=200 合法，夹到 100 后累计；编号 1 的 -5 合法，贡献 0。后来的 {2,10} 是重复，{-1,30} 因编号非法计 invalid。第二组的 -1001 非法且不占编号，30 成为首次合法值；最后的 1001 仍应计非法，不能因为编号已出现就改计重复。

B1　挑战：有符号十进制解析　*(+10 分)*

背景

比赛日志里最难找的错误，有时只有一个数字。你以为接收到的是 2147483647，转换以后却变成了负数；界面没有报错，下游就把一个巨大正值当成了紧急反向指令。
通信组想要一个范围清晰的有符号整数解析器。它必须在危险的乘法或加法发生之前拦住溢出，而不是先算坏再检查。标准转换库当然可以解决很多实际问题，本挑战刻意不让你调用它们，是为了让你亲自处理符号、数字、边界以及失败时的输出保护。这是可选拓展，不会成为必做题的前置条件。

你的任务

实现 `parse_i32`。返回 bool 表示转换是否成功；`output` 是由调用方提供的输出变量，只有成功时才允许改写。若尚未学习引用参数，可先按骨架把它理解为“成功后交回的结果位置”。

考点：字符处理、引用输出、溢出预防（选做）。实现文件：`src/main/A1_b1.cpp`。

固定接口与数据含义

```
bool parse_i32(const std::string& text, int& output);
```

接口位于骨架已有的 `aim::b1` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. 字符串长度 ≤ 1000；合法语法为可选单个 + 或 -，随后至少一位 ASCII 数字 0..9。
2. 允许前导零、+0、-0；禁止任何空白、小数点、第二个符号、非 ASCII 数字或尾随字符。空串与单独符号非法。
3. 结果必须在 [-2147483648,2147483647] 内，包括两个端点；不能假设负数范围与正数范围完全对称。
4. 成功时写入 output 并返回 true；任意失败返回 false，且 output 保持调用前的值。
5. 禁止 stoi/stol/stoll、字符串流及 C 数字转换函数；逐字符检查，并在可能溢出的运算之前拒绝输入。
6. 要求 O(字符串长度) 时间、O(1) 额外空间，不使用自定义 class。

样例与逐步说明

```
int value = 9;
parse_i32("-2147483648", value); // true; value=-2147483648
parse_i32("2147483648", value);  // false; value remains -2147483648
parse_i32("-0000", value);       // true; value=0
```

第一条正好是最小 32 位有符号整数，合法。第二条比最大正数多 1，必须失败，并保留上一次合法结果。第三条说明前导零不是错误，负零按普通零返回。

B2　挑战：原地稳定压缩　*(+10 分)*

背景

一段传感器缓存里混着离群值，通信线程只想把可信区间内的数据压到前面，再把新的有效长度交给下一段程序。排序会破坏时间顺序，复制一份数组又会多占一块内存。
这次你拿到的不是一个可以随意增长的容器，而是一段已经存在的整数数组和它的长度。你不能释放它、替换它或偷偷新建第二份缓冲区，只能在原地整理。留下来的数据仍按采样先后排列。题目放在挑战区，是为了给已经接触指针和数组传参的同学多一步练习；它不包含资源所有权或面向对象设计。

你的任务

实现 `compact`，保留闭区间 [low,high] 内的元素，将它们稳定地移到数组前部，并返回保留数。后续调用方只读取新长度之前的前缀；尾部不是结果的一部分。

考点：数组指针、原地处理、稳定性（选做）。实现文件：`src/main/A1_b2.cpp`。

固定接口与数据含义

```
std::size_t compact(int* data, std::size_t n, int low, int high);
```

接口位于骨架已有的 `aim::b2` 命名空间内，不要再嵌套一层。普通 struct 已由骨架提供，只含数据字段，不要重复定义。

规范

1. $n \le 200000$；$n=0$ 时允许 data=nullptr，此时不能解引用。$n>0$ 时保证 data 指向 n 个合法 int 元素。
2. 保留满足 $low \le data[i] \le high$ 的元素，两端都包含，重复值也照常保留。
3. 稳定性指保留元素之间的相对顺序不变，禁止为了方便而排序。
4. 返回 k 后，data[0..k-1] 必须是正确结果；data[k..n-1] 的值不规定。
5. low>high 时返回 0，且整个数组不修改；全部不命中时也返回 0。
6. 要求 O(n) 时间、O(1) 额外空间，禁止辅助容器或新数组，不释放调用方数组。

样例与逐步说明

```
int data[] = {-1,2,0,9};
std::size_t k = compact(data,4,0,2); // k=2; prefix {2,0}
compact(nullptr,0,0,1);              // 0
```

原数组中的 2、0 在区间内，必须按这个先后次序出现在前两格，不能排序成 0、2。新的长度是 2；不要对后两格的残留值作任何假设。空数组只返回 0，不需要也不能读取 data[0]。

提交与验收

开发与提交

在学生 template 中开发，保留 src/main 下各题文件名。CPP-Classroom 运行后还需主动提交；教师按学生导出源码，运行成功不会自动替代提交。使用 GitHub 仓库的同学在 main 分支持续开发，最终交仓库链接和最终 commit；平台提交者交平台记录及教师要求的源码导出。具体截止日期由教师在课堂或通知中公布，不从题面编译日期推算。

只修改 src/main 中的实现区与 README 末尾的个人说明；另允许提交自动生成的.agent-sessions/bundles/ 归档。自己的实验测试放在新建的 local_tests 目录，不改公开测试、固定接口、标记、工具和工作流。按题或按缺陷保留可审阅的增量记录，不要求提交未完成的本地实验。提交前重新获取干净模板，把自己的实现放回去跑一次。

README 为每题记录思路、关键边界和复杂度，写出至少三个自己补充的边界测试，并说明一处实际发现的缺陷、定位方法和修复。允许使用 AI，须如实说明使用范围、如何核对生成内容，并保留能复核的过程记录；未使用 AI 时明确声明并提供开发记录。能解释自己的接口与失败路径是验收的一部分。

AI 使用与 Agent 会话归档

允许并鼓励合理使用 AI 辅助开发。是否使用 AI 本身不影响功能分；你仍须核对代码并解释自己的接口、状态更新与失败路径。AI 使用透明度占人工部分 20 分，以声明、归档覆盖度、真实性和核对说明共同评阅。未使用 AI 的同学明确声明并保留开发记录，不需要伪造会话归档。

参照 Python Assessment，使用 Agent Session Commit（AgentLedger）固定版本 0.1.5 记录过程。归档步骤在本地执行，需要 Git 和 Python 3.10+；C++ 程序本身仍只依赖 C++17。在各自 Assessment 的个人学生 Git 仓库内配置，不要在教师完整目录或其他项目里安装。即使使用 CPP-Classroom 编码，也应将最终实际提交的源码同步到个人仓库，随仓库提交增量记录和归档。

在长期保留的 Python 环境中安装，进入学生仓库根目录后运行：

```
python -m pip install --index-url https://pypi.org/simple 'agent-session-commit[pre-commit
    ]==0.1.5'
agent-session-commit install --pre-commit
agent-session-commit status
```

安装器让你选择实际使用的 Agent 并确认会话来源，配置只对当前 clone 生效。每份 Assessment、每次新 clone 都单独配置；升级后重新运行安装器刷新 hooks。随模板提供的.pre-commit-config.yaml 已将归档后端固定为 v0.1.5，不要改成旧版。保持安装时的 Python 环境可用，移动或删除环境后须重新安装 hooks。

每次正常 git commit 成功后，post-commit hook 将新增会话打包到.agent-sessions/bundles/，再 amend 到刚完成的同一个提交；没有新增会话时不另建归档，也不产生额外的归档 commit。amend 会改变提交 SHA，最终提交号以 git rev-parse HEAD 为准。只有 status 正常不能证明记录完整：完成一次真实修改和提交后，再检查最终提交中是否包含对应归档。

```
git rev-parse HEAD
git show --stat HEAD -- .agent-sessions/
```

选择的 Agent 或会话目录变化时重新运行安装器；浏览器聊天、未支持的 IDE 或远程环境不能假定会自动收集，应按工具支持的格式导出实际会话并选择 Custom 来源。原始导出可放.agent-sessions/source/（已忽略），但.agent-sessions/bundles/ 必须随 Git 提交，不能把整个.agent-sessions/ 加入忽略列表。README 的“AI 使用与开发过程”要说明使用的工具、范围、归档位置、如何核对结果，以及未归档部分的原因和补充材料。

归档是普通 Git 内容，不加密、不会自动脱敏，推送到公开仓库后他人可读取。确认所选来源属于本作业，不要把口令、令牌或无关项目的私密会话放进去。没有使用 AI 时如实声明；记录不完整时如实说明，不编造、替换或回填成不存在的对话。

课程会结合会话归档、仓库内标记检索和 Git 历史分析核对 AI 使用情况。透明使用 AI 不扣分；单一标记不能替代对完整开发记录的复核，也不会单独决定功能分或最终总分。

评分

Q1–Q8 的原始权重依次为 8、10、10、12、12、14、16、18，合计 100，按 0.6 折为自动功能分 60。每题按固定测试组通过率给部分分；随机组内全部断言通过才算该组通过。B1、B2 各加 10，不按 0.6 折算，奖励最多 20。

人工部分共 40：过程透明度 20（使用声明完整性 5、开发/辅助记录可复核性 10、对结果的核对说明 5）；代码质量与说明 20（清晰命名与组织 5、遵守实现约束和复杂度 5、边界自测与缺陷分析 5、设计解释 5）。未使用 AI 的同学按开发记录的同等可复核程度评分，不因未使用 AI 扣分。常规部分满分 100，含挑战总上限 120。

公开全过不代表隐藏全过。缺文件、TODO、接口不匹配、编译失败、未要求的异常、超时、伪造输出和测试未到完成点的提前退出均不算通过；单题失败不阻断其他题。grader 的人工分字段留空、总分为 null，待教师评阅后汇总，不把自动 60 分当成最终成绩。

资源与复核

正式 grader 每题编译墙钟 45 秒、每测试组运行墙钟 5 秒，Linux 运行地址空间 512 MiB。题目给出的复杂度和禁止使用的实现方式属于人工复核范围，单靠一次运行变快不能证明达标。CPP-Classroom 自身限额可能不同，正式判分以教师统一环境为准。

发现题面、骨架和测试有冲突时，保留最小复现、实际结果与对应条款，向教师反馈；不靠修改测试或猜测隐藏要求解决冲突。